

Authorization Receipts: Offline-Verifiable Evidence of Named-Human Authorization for Irreversible AI-Agent Actions

Iman Schrock EMILIA Protocol, Inc. team@emiliaprotocol.ai

Preprint. July 2026.

The protocol described here is specified in a set of active individual Internet-Drafts (draft-schrock-ep-authorization-receipts-05, draft-schrock-human-authorization-binding-00, draft-schrock-ep-action-evidence-graph-00, and companions). These are individual submissions to the IETF Internet-Draft repository; they are not IETF-adopted or endorsed documents, and nothing in this paper should be read as implying working-group status.

Abstract

AI agents increasingly hold credentials sufficient to perform irreversible operations: releasing payments, changing beneficiary records, rotating production credentials, deleting data. When such an action is later disputed, the available record is typically a row in a database controlled by the operator of the approval workflow, which is to say, testimony from the party whose conduct is under examination. This paper describes the authorization receipt, a cryptographic artifact that converts the fact "a named, accountable human approved this exact action, exactly once, before it executed" from testimony into evidence. An approver holding their own signing key signs a canonical authorization context binding the action's content hash, a policy commitment, a one-time nonce, and a validity window; the enforcement point consumes the authorization at most once; and the receipt carries its own Merkle inclusion proof against a signed log checkpoint, so a relying party can verify it with no network access to the operator, the log, or any API. We describe the threat model, the receipt and quorum constructions, a host-agnostic binding profile for carrying the evidence in other record formats, and a purpose-relative evidence-sufficiency layer with deterministic, replayable verdicts. The core state machine and quorum construction are machine-checked in TLA+ and Alloy; we state precisely what those models do and do not cover. Verification establishes signature, binding, and log-inclusion integrity; it never establishes that the authorized action was correct, and we discuss the residual risks, including presentation attacks, that cryptography narrows but does not remove.

1. Introduction

1.1 The testimony-versus-evidence problem

Consider a dispute that is now routine in outline and will shortly be routine in fact. An AI agent operated by a financial-services firm releases a wire transfer. The transfer is later challenged, and the firm produces its approval records: a workflow-tool entry showing that a user clicked "approve" at a certain time.

Every element of that record is an assertion by the firm. The database holding the approval row is writable by the firm's administrators; the mapping between the row and the executed action is maintained by the firm's software; the timestamp is the firm's clock. An auditor, counterparty, insurer, or court that wants to

rely on the record must trust the operator of the approval system, and the operator is frequently the party whose conduct is in question. The record is testimony.

Existing authorization infrastructure does not change this, because it answers a different question. Identity and access management establishes that an actor is authenticated and authorized *in general*: it grants sessions and scopes, not decisions about individual actions. Fraud that occurs inside a valid session through approved channels, such as a business-email-compromise-driven beneficiary change, is invisible to session-level controls. Where human approval does exist, it is a click recorded in a mutable store. Three gaps follow, and they are structural rather than implementation accidents:

1. **The action gap.** Authorization is granted to sessions and scopes, not to individual actions with concrete parameters.
2. **The accountability gap.** No independent cryptographic evidence binds a specific human to a specific action; the approval record is producible and alterable by the operator.
3. **The verification gap.** Third parties must trust the operator's logs; no artifact exists that they can check with mathematics alone.

The authorization receipt is a narrow construction aimed at exactly these gaps: before an irreversible action executes, a named approver signs the exact action with a key only the approver holds; the signed authorization is consumed exactly once; and the resulting receipt is verifiable offline, indefinitely, by any party holding the receipt, the approver key material, and a log checkpoint that travels inside the receipt itself.

1.2 What this paper claims, and what it does not

The claim is deliberately limited. A receipt is evidence that a key enrolled under a named approver identifier signed the canonical bytes of one action, under a committed policy version, within a validity window, and that the authorization was consumed once. It is not evidence that the action was a good idea, that the approver understood the business context, or that the surrounding deployment cannot be bypassed. Verification proves signature, binding, and log-inclusion integrity; it never proves business correctness. The organization publishing this work is not an auditor, regulator, or insurer; the artifacts described here support the judgments of such parties and conclude nothing on their own.

1.3 Contributions

- A threat model for named-human authorization of agent actions in which the orchestrating operator is untrusted for evidence purposes (Section 2).
- The receipt construction: canonical action bytes, approver-held keys, one-time consumption as enforcement-point state, and an offline verification algorithm whose log checkpoint travels inside the artifact (Section 3).
- A quorum construction whose unit of distinctness is the accountable human, not the signature (Section 4).
- A host-agnostic binding profile (five requirements, B1 through B5) for carrying the evidence in other agent-action record formats without redefining it per host (Section 5).

- An evidence-sufficiency layer: content-addressed evidence graphs, relying-party-supplied policies, a closed five-verdict set with fixed precedence, and deterministic replay (Section 6).
- Machine-checked models of the core state machine and quorum, with an explicit statement of scope (Section 7), and a cross-language conformance suite (Section 8).

2. Threat Model

Three adversarial settings drive the design. They are treated together because a mechanism that addresses one while silently assuming away another produces evidence that fails exactly when it matters.

2.1 Compromised or malicious operator

The party running the orchestration service (policy registry, signoff routing, log) is not trusted for evidence purposes. Under the recommended key custody classes (Section 3.5), a compromised operator can deny service and can fail to route signoff requests; it cannot forge an approver's signature, because it does not hold approver keys, and it cannot replay a genuine one, because nonces are single-consumption and receipts chain.

Two operator-compromise paths remain open and are part of the model rather than claimed away. First, an operator that controls the signing client's rendering can harvest a *genuine* signature over an action the approver misunderstood; this is the presentation-attack family of Section 10.2, and it is why independently authored rendering surfaces are required for high-value policies. Second, an operator that unilaterally controls the directory of enrolled approver keys can enroll a key it controls under a legitimate approver's name, relocating the forgery rather than preventing it; Section 3.5 describes the directory-authority controls. The accurate statement of the property is therefore: the operator cannot forge an approver's signature. The stronger statement, that the operator cannot obtain an unauthorized approval at all, additionally requires the directory-authority and independent-rendering controls to be deployed.

2.2 Compromised agent

The initiating agent is identified but never trusted. A prompt-injected or otherwise compromised agent can propose any action, state any escalation reason, and attribute itself to any external identity. The design consequence is uniform: everything the initiator supplies is treated as a claim. Injection can change what the initiator *proposes*; it cannot change what a human *approves* on the human's own hardware, because the device-bound signature is produced outside the model's context. The protocol additionally enforces separation of duties: an initiator can never occupy an approver slot for its own action.

2.3 Post-hoc dispute

After execution, the parties evaluating the evidence (auditors, counterparties, insurers, courts, regulators) may distrust every online service involved, may be examining events years old, and may have no network path to the original operator, which may no longer exist. The design consequence is offline verifiability: a receipt must be checkable using only its own bytes, pinned or directory-proven approver key material, and a log checkpoint carried inside the receipt. What offline verification can and cannot establish in this setting is bounded precisely in Sections 3.4 and 10.3.

2.4 Out of scope

The protocol binds approvals to *approver identifiers* whose keys are enrolled in a directory; proving that the holder of an identifier is a particular natural person is the job of the identity-proofing layer that populates the directory, not of the receipt format. Collusion among distinct enrolled humans, one human controlling multiple enrolled identities, and coercion of an approver are likewise not defeated by the mechanism; receipts make such events attributable, which raises their cost, and no more than that.

3. The Receipt Model

3.1 Actions as canonical bytes

An action is a single proposed operation with concrete parameters: one wire transfer to one beneficiary for one amount, one credential rotation of one key. It is represented as a JSON Action Object containing at minimum the action type, target system and resource, parameters, initiator identity, policy reference, and request time. The object is serialized under the JSON Canonicalization Scheme (JCS, RFC 8785), and the *action hash* is the SHA-256 digest of the canonical serialization. Implementations must reject an approval request whose action hash does not match a locally recomputed hash of the presented object. Sensitive parameter values may be carried as salted hashes, provided the executing system can recompute them; the binding property is preserved because the hash commits to the committed values.

Canonical bytes are the foundation of every later property: an approval bound to a description or a workflow-ticket identifier binds to something the operator can re-narrate; an approval bound to the SHA-256 of canonical bytes binds to exactly one action.

3.2 The authorization context and the signoff

For each required approver, the orchestrator constructs an Authorization Context: a canonical JSON structure containing the action hash, the policy identifier *and* a policy hash committing to the exact policy version evaluated, the initiator identity, the approver identifier and slot index, the required approval count, a one-time nonce of at least 128 bits of CSPRNG output, issuance and expiry times, and a hash chaining the attempt to the issuing log's most recent receipt. The context is JCS-canonicalized; the *context hash* is its SHA-256 digest; the approver signs the context hash.

Two rules deserve emphasis. First, the policy commitment is exact: a signature over a context with policy hash X must not satisfy a requirement evaluated under policy hash Y, even for the same policy identifier. Approvals cannot migrate across policy versions. Second, the approver must be shown, at signing time, a faithful human-readable rendering of the Action Object, not only its hash; interfaces that display a different action than the one hashed constitute a presentation attack (Section 10.2).

The context may optionally carry two further members, both claims by parties the protocol identifies but never trusts. An *initiator attestation* records the initiator's own stated reason for escalating to a human, as a closed enumeration (irreversibility, magnitude, uncertainty, novelty, authority gap, policy rule) plus an optional rule identifier and a length-capped free-text statement. An *agent binding* attributes the action to an external agent identity and, optionally, the external delegation under which the agent acted, with an optional observed-at timestamp against which a relying party may enforce evidence freshness, fail-closed. Because

JCS serializes every member present, both are covered by the approver's signature: the receipt proves the stated reason and claimed attribution were part of what the approver signed. Neither is proof of the initiator's internal state, the agent's identity, or the delegation's validity, and policy engines must not use attestation content to relax any threshold: the initiator must gain nothing by saying the right words.

The signoff itself carries the context hash, the signature, the key custody class, the approver key identifier, and, for device-bound keys, the WebAuthn assertion material. For device-bound keys the WebAuthn challenge must equal the context hash and the authenticator's user-verification flag must be set, so the signature attests both possession of the enrolled authenticator and a local user-verification event. Denials are signed over the same context hash with a denial envelope, so refusals are equally non-repudiable and equally terminal: "an accountable human refused this action" is a positive, verifiable fact, not an absence.

3.3 One-time consumption as enforcement-point state

An authorization attempt moves through a small state machine:

```
REQUESTED -> {PARTIALLY_APPROVED}* -> APPROVED -> COMMITTED
          \-> DENIED                      \-> EXPIRED
          \-> EXPIRED
```

COMMITTED, DENIED, and EXPIRED are terminal. The protocol invariants, maintained as machine-checked models (Section 7) and required of conforming implementations, are:

- **ConsumeOnce.** A nonce transitions to a terminal state at most once, globally; any second presentation is rejected as a replay.
- **BindingMatch.** A signoff satisfies only the context, and therefore only the action hash, that it signs.
- **TerminalIrreversibility.** No transition exits a terminal state.
- **SelfApprovalImpossible.** For every signoff, the approver differs from the initiator; under multi-approver policies, approvers are pairwise distinct and each distinct from the initiator.
- **NoBypassWrite.** A COMMITTED state is reachable only through the full sequence, and a conforming verifying executor does not execute without verifying it.

The consumption record is enforcement-point state, not log decoration. An authorization, once consumed or refused, is terminally unusable; replay across sessions, operators, or time is detectable and must be rejected. This is what separates a receipt from a bearer credential: possession of a receipt confers no authority to execute anything, because the underlying authorization has already been consumed by exactly one execution.

Deployment topology is graded honestly rather than assumed. In the strongest conformance class, the system of record itself (payment switch, registry, deployment controller) verifies the authorization bundle before executing and refuses otherwise. A middleware class intercepts between the agent and the executing credential, which defends against agent error and prompt injection but can be bypassed by an operator with code control. An evidence-only class makes no enforcement claim at all. Implementations must declare their class in the receipts they produce and must not state a stronger class than deployed; the distance

between "the protocol has these properties" and "your deployment is unbypassable" is the most common overclaim in this category.

3.4 The trust receipt and offline verification

Upon commitment, the orchestrator assembles the Trust Receipt: the full Action Object and its hash, every authorization context, every signoff, the consumption record, a Merkle inclusion proof for the receipt leaf against a signed log checkpoint (tree size, root hash, log signature, log key identifier), and inclusion proofs for the approver key entries in the approver directory. Operator-produced commitments and receipt-log material are signed with Ed25519; approver signoffs under the device-bound class are ES256 (P-256) or Ed25519 where the authenticator supports it.

A verifier holding the receipt, a trusted log public key, and a trusted directory root or pinned approver keys, with *no network access*, establishes six things: (1) the action hash recomputes from the canonical Action Object; (2) each context hash recomputes and commits to the action hash, the policy hash, and a distinct approver; (3) each signature, including the WebAuthn assertion where present, verifies over the context hash against a key entry whose validity window contains the issuance time; (4) separation of duties holds and the approval count satisfies the policy; (5) the Merkle inclusion proof verifies against the checkpoint root and the checkpoint signature verifies against the log key; (6) signing and commitment times fall within the validity window.

Step (5) is what distinguishes this design from log-access designs: the checkpoint travels inside the receipt, so verification requires no query to the log.

One further property is deliberate and load-bearing: no step of offline verification relies on a message authentication code, a shared secret, or any symmetric primitive. A symmetric construction, such as an HMAC-chained audit log, is verifiable only by a party holding the same secret as the producer, precisely the party the evidence is meant to constrain; its keeper can rewrite its own history undetectably. Every artifact on the verification path is bound by an asymmetric signature whose verifying key the relying party holds independently of the issuer.

Equally important is what offline verification does *not* establish, and the specification requires implementations to say so. It establishes authenticity as of commit time, not currency: a receipt whose approver key was revoked an hour after commitment still verifies, because the artifact is evidence of validity at commit time. And it establishes inclusion in *a* tree whose head the log operator signed, not that this tree is the only tree the operator has shown the world; split-view detection requires online activity such as gossip or witness cosigning. A relying party with freshness or revocation requirements must additionally consult a current directory head and checkpoint online. Implementations must not describe offline verification as establishing that a receipt is "currently valid."

3.5 Key custody and the approver directory

Key classes classify custody of the approver's signing key and nothing else. Class A keys are generated and held in a platform authenticator or security key and exercised via WebAuthn with user verification required; this is the recommended class. Class B keys are software keys in the approver's client environment, acceptable where WebAuthn is impractical, with the reduced assurance noted in receipts. Class C, in which

the operator signs on the approver's behalf after authenticating them, exists only to describe pre-existing deployments; receipts produced under it must be labeled as such, and relying parties should treat them as operator assertion, not approver signature.

Approver public keys are enrolled in a signed, Merkle-tree-structured approver directory maintained per organization, and a receipt carries an inclusion proof of the relevant key entry, so verification needs no live directory access. Directory authority is itself a trust root and must not default to the operator: the directory tree head must be signed by an organization-controlled key, and where directory *operation* is delegated to the operator, every enrollment must carry a second-party attestation by an organization administrator key or a quorum of already-enrolled approvers. A verifier presented with a directory head signed only by an operator-held key treats the whole structure as operator assertion, equivalent in assurance to Class C, regardless of the custody class of individual signoffs. The rationale is the relocation argument of Section 2.1: an operator that cannot forge signatures but can decide which keys count has not been removed from the trust path.

The directory is also the explicit slot where identifier-to-person binding lives. The receipt format proves that a key enrolled under a given identifier signed the exact action; the strength of the claim that the identifier names a particular human is a property of the directory authority and whatever identity-proofing layer feeds it, and it is deliberately not smuggled into the receipt's own claims.

4. Quorum and Distinctness

High-consequence actions frequently require more than one approver, and multi-party approval introduces a failure mode that single-approver analysis misses: counting signatures instead of humans.

Under an m-of-n policy, each required approver receives and signs an individual authorization context sharing the same action hash and nonce family but carrying a distinct approver index. Commitment occurs only when the required number of valid, distinct signoffs exists before expiry. Partial approval confers no authority whatsoever: a verifying executor presented with fewer than the required signoffs refuses.

The distinctness requirement is stated at the level that matters: the required number of *distinct accountable humans*, not merely distinct signatures or distinct keys. The corresponding machine-checked properties (Section 7) include not only SelfApprovalImpossible but also the properties that no human fills two quorum slots and no key fills two quorum slots, which together rule out the two standard degenerate quorums: one person with two credentials, and one credential presented twice. Where a policy requires ordered approval, the chain is checked for acyclicity and linearity.

One residual vector in the multi-approver setting survives all signature checking and is stated plainly. Since each approver signs their own context, a malicious orchestrator can show different approvers different optional-member content (for example, different initiator attestations), and every individual signature remains valid. The specification therefore requires cross-context consistency: an optional member present in any context of a receipt must be present, byte-identical in canonical form, in every context, so every approver demonstrably signed the same stated reason and attribution. Verifiers implementing the members flag violations; the rule is a conformance check layered above signature validity, not a change to it.

Delegated approval authority is constrained rather than prohibited: a delegation record must appear in the receipt's key proofs, a delegate's effective scope is the intersection of the delegation grant and the principal's authority at signing time, chains are bounded in depth, and every link is independently signed.

5. Host-Record Binding

Receipts do not live alone. The agent-action record formats now in development (post-execution capsules, pre-execution permits, audit-trail records, provenance graphs, intent chains) almost uniformly make the same, sound scoping decision: human authorization is somebody else's format. The result is a set of reserved-but-empty slots: an approver disposition whose authority is opaque, an authority-context stub, a human-override field with a privacy carve-out where the human should be, a "signed grant" whose format is an unassigned work item. A dozen slots with a dozen ad-hoc answers would be worse than none.

The binding profile defines, host-agnostically, what filling such a slot means. Evidence is bound either **by reference** (a member carrying the SHA-256 digest of the authorization artifact's canonical bytes, an artifact-type token, and an optional locator hint) or **embedded** (a compact self-describing claim carrying the action digest, mode, named approvals with each approver's own signature object, and optional quorum semantics). Five requirements make the binding mean the same thing in every host:

- **B1, digest grounding.** A binding is credited only against artifact bytes: the reference form by digest equality, the embedded form by verifying the contained signatures. A host field that merely asserts "a human approved," with neither, must not be treated as human-authorization evidence.
- **B2, action agreement.** When the host record binds an action, the authorization artifact's action binding must agree with it. An artifact authorizing a different action invalidates the binding; it does not merely weaken it. This defeats splicing a genuine artifact from one action into another's record, the confused-deputy case.
- **B3, verified versus accepted.** Verifying a binding (digests and signatures hold) is distinct from accepting it (the relying party trusts the artifact's issuer via out-of-band pinned key material). A verifier must report the two separately, and a binding from an unpinned issuer must not be accepted; a self-issued artifact cannot be self-accepted.
- **B4, fail-closed absence.** The absence of a binding is the absence of evidence, never a default. A relying party whose policy requires human authorization treats an unbound or unresolvable binding as insufficient. Absence becomes positive evidence only through a signed *observed-absence* statement: an attestation that the verifier performed a defined search against defined sources at a stated time and found none, attesting the search and its emptiness, never a universal negative.
- **B5, consistency.** When a host record carries both forms, the embedded claim's canonical bytes must hash to the reference's digest; a mismatch invalidates both. The two forms may not tell two stories.

For transparency-logged signed statements (the SCITT host family, RFC 9943), the profile fixes the digest choice per host-statement profile: either a digest over the receipt payload's canonical bytes, for offline composition, or a digest over the COSE_Sign1 bytes of the receipt expressed as a signed statement, recommended when the receipt is registered in a transparency service so the reference also resolves to a logged, inclusion-proofed entry. The reference form additionally serves disclosure minimization: the named

human travels only in the artifact, which can be withheld until a relying party with a need to know requires it, at the cost, by B4, of the evidence not counting until produced.

Two boundary rules complete the profile. A host record must not extend an artifact's validity by carrying it; replay and one-time-use semantics belong to the artifact and its enforcement point. And the profile defines no authorization format of its own: a bound artifact verifies under its own specification.

6. Evidence Sufficiency: the Action Evidence Graph

6.1 The relying party's question

The standards landscape now produces many signed artifacts about one agent action: workload identity credentials, delegation and grant tokens, call-chain transaction tokens, runtime attestation results, pre-execution permits, named-human authorization receipts, post-execution records, transparency-log inclusion receipts. Each answers its own question. None answers the relying party's: given all of these artifacts, is this action's evidence sufficient *for this reliance purpose*? A bank releasing a wire, an insurer paying a claim, and a regulator auditing an agent need different sufficiency bars over the same artifacts, and today each such decision is ad hoc, non-reproducible, and leaves no evidence of its own.

6.2 Content-addressed graphs, edges as claims

An Action Evidence Graph is a portable JSON document carrying the action digest, nodes, and edges. Each node's identifier is the SHA-256 digest of the referenced artifact's JCS-canonical bytes, with a type token and, optionally, the artifact inline; a verifier rejects an inline artifact whose canonical digest does not equal its node id. Because nodes are references, a presenter can disclose the *shape* of its evidence without the contents; an undisclosed node contributes nothing to sufficiency, and a required undisclosed node fails closed.

Edges are presenter claims, and this framing carries the security weight of the layer. An edge (for example, "this permit *permits* that authorization receipt") is credited only if the claimed binding is present in the source artifact's own canonical bytes, at minimum containment of the target's digest. A claimed edge that the bytes do not back *poisons* the evaluation: the graph has asserted something its evidence does not support, and the verdict must not be admissible.

Graph identity is disclosure-independent: the graph digest is computed over the sorted (id, type) node pairs, the sorted edges, and the action digest, and never over inline artifact bytes, so two parties holding different disclosures of the same graph agree on what graph they are discussing.

6.3 Policy replay and the closed verdict set

The sufficiency bar is an evidence policy supplied by the relying party. The graph document has no policy member, by construction: a presenter choosing its own sufficiency bar is the confused-deputy failure of evidence systems. A policy names its reliance purpose, a boolean requirement expression over node types, per-type freshness bounds, per-type revocation requirements, and required edges that must be present and byte-backed.

Evaluation is deterministic and offline: given the same graph, policy, and evaluation time, any implementation reaches the same verdict and the same replay digest. The verdict is one of a *closed* set of five: `admissible`, `missing_evidence`, `stale`, `conflicted`, `unverifiable`, with precedence **unverifiable over conflicted over stale over missing_evidence over admissible**, so no failure ever degrades toward admissibility. Machine-readable reason codes accompany the verdict; the reason vocabulary is open, the verdict set is not.

The failure semantics encode a deliberate distinction between absence and deception. A node referenced but not disclosed contributes an unverified fact; if the policy required that type, the verdict is `missing_evidence`: the evidence may exist but was not presented, and nothing about the graph lied. A claimed edge the source bytes do not back is a lie about the evidence and forces `unverifiable` regardless of what else verified. Absence degrades to "not enough"; deception degrades to "not trustable."

The replay digest is the canonical digest of exactly three inputs: the policy as supplied, the ordered list of normalized evidence facts (per node: type, label, verified flag, attested action digest, outcome if any, revocation flag, age, derived staleness, and whether a revocation check was required), and the evaluation time, bound to the graph digest. Raw artifact bytes, network state, and clock reads are excluded by construction; two evaluators that disagree on a replay digest are, by definition, evaluating different inputs.

6.4 Reliance results and policy packs

The verdict may itself be issued as a signed artifact carrying the verdict, reasons, action digest, graph digest, policy digest and identifier, reliance purpose, replay digest, and evaluation time, signed by the relying party's verifier key. The signature adds accountability, never authority: any third party can recompute the replay digest rather than trust the signer, and the verified/accepted distinction of B3 applies to the result itself. Signed reliance results make reliance decisions auditable: an institution can prove, later, which policy it applied to which evidence before it acted.

Six policy packs profile the mechanism for concrete irreversible action classes: wire transfer, vendor bank-account change (a distinct-human quorum), credential rotation, production data deletion, regulated trade execution (post hoc: the execution record must provably reference its authorization and be transparency-logged), and healthcare record export. A pack is a starting point the relying party adopts and owns, pinning its own trust anchors and tightening freshness to its risk appetite; no pack waives fail-closed behavior.

A verdict of `admissible` is evidence that a bundle met a stated policy at a stated time. It is not adjudication, it does not establish that the action was correct or safe, and it confers no authority beyond what the underlying artifacts carry. The token is a protocol token naming sufficiency under the stated policy; it carries no legal meaning.

7. Formal Analysis

The protocol's core state machine is maintained as a TLA+ model checked with TLC (version 2.19). The checked configuration explores the handshake lifecycle, signoff flow, delegation, revocation/consumption races, and identity-continuity extensions, generating 413,137 states (45,342 distinct) with no counterexample against 26 invariants. These include the properties an evidence artifact stands on: an

authorization is never consumed twice; a consumed authorization is never subsequently revoked, and a revoked one never consumed; the lifecycle is acyclic and terminal states are inescapable; nonces are unique; delegation chains are acyclic and delegates cannot exceed their principals' authority; direct-write bypass of the state machine is impossible; concurrent revoke/consume interleavings serialize; and replay or duplicate consumption attempts are rejected. Model checking earned its keep during development: TLC surfaced four real specification bugs, all fixed before the properties passed.

The quorum construction is separately modeled in Alloy: a checked model proves `SelfApprovalImpossible`, `NoHumanFillsTwoSlots`, `NoKeyFillsTwoSlots`, `TwoPersonRuleHolds`, and acyclicity/linearity of ordered approval chains against the quorum verifier and its conformance vectors. Additional Alloy models cover the protocol's relational structure and federation behavior.

The honest scope statement matters as much as the property list, and the specification makes it normative rather than leaving it to marketing judgment:

- The models cover the core authorization state machine and the quorum construction. They do **not** cover the `WebAuthn` challenge binding, the approver directory, log checkpoints, or the optional initiator-attestation member. Those parts of the specification are specified, not proven, and extending the models to them is tracked work.
- TLC's result is model checking at small scope (a single handshake and single claim in the checked configuration), which covers all per-handshake and per-claim safety properties; it is exhaustive within those bounds, not a general proof for unbounded instances.
- The models prove safety of the modeled state machine. They prove nothing about any AI model's behavior, about host compromise, or about deployments in a weaker conformance class, and implementations must not represent the proofs as covering deployment topologies they do not model.
- Three normative mechanisms in the current specification are ahead of the reference implementation and not yet exercised by it or by conformance vectors: the operator-signed-directory assurance downgrade, delegation records in the receipt's key proofs together with the delegate-cannot-exceed-principal check, and enforcement-class emission. The specification directs implementers to treat the text as normative and the reference implementation as incomplete on these points, not the reverse.

The distinction the project's own status tracking draws is the right one to repeat here: *specified* means a property is written down as a theorem; *verified* means a model checker has run and found no counterexample under the model's assumptions. The two are conflated in this field often enough that keeping them separate is itself a contribution to hygiene.

8. Implementation and Conformance

A reference implementation of the receipt, quorum, evidence-chain, binding, and evidence-graph layers is published under Apache-2.0, together with a conformance battery of 8 suites comprising 58 vectors. The receipt vectors are self-contained (each carries its own public key and document, requiring no server and no shared state), adversarial (reject vectors each pin one invariant: tampered bytes, wrong key, replay, malformed signature, broken Merkle anchor), and deterministic (regenerable byte-identically from fixed

seeds). Further suites cover quorum semantics, signoffs, evidence records, provenance chains, revocation timing, execution integrity, and an optional JWS serialization that standard JOSE libraries can verify.

The independent-implementation status is stated precisely, because it is the most tempting place to overclaim: the reference verifiers are JavaScript, Python, and Go in one repository, a consistency check, not independent implementations; an independent clean-room reimplementation (COSA) is underway. The clean-room effort, announced on 2 July 2026, reimplements the receipt and quorum verification path directly from the specification, to be validated against the published conformance vectors in the reimplementer's own repository; at this writing that work is announced and in progress, not complete, and its result will be reported either way. Separately, a third party has independently executed and verified the published artifacts against a pinned commit.

The evidence-graph layer ships with a test suite covering the fail-closed invariants of Section 6, including disclosure-independent graph identity, unbacked-edge poisoning, required-edge stripping, freshness degradation, and replay determinism, together with a complete worked vector (graph, relying-party policy, and signed reliance result) reproducible from a fixed seed. Deterministic vectors for the binding profile's checks and for the observed-absence statement are published alongside.

9. Related Work

The receipt is positioned as a composition with adjacent layers, not a replacement for any of them; each answers a different question, and the composition is the point.

Transparency and logging. SCITT (RFC 9943) provides an append-only transparency log with inclusion receipts, and is deliberately agnostic about *who authorized* a statement, which is precisely the question the authorization receipt answers. The two compose directly: a receipt can be expressed as a COSE signed statement and registered in a transparency service, and the binding profile's statement-digest form (Section 5) is designed for that carriage. The agent-action statement cluster emerging around SCITT (capsules, permits, post-execution records, refusal events) makes agent actions transparent, logged, and policy-checked; those profiles reserve slots for, and do not produce, the named-human authorization evidence itself. Receiver-attested logging has the receiving service sign what it observed, post hoc; pre-execution authorization and post-execution attestation are complementary halves of a complete record.

Workload and agent identity. WIMSE workload credentials authenticate which workload is calling and prove key possession; their chartered scope is workload identity, and they deliberately do not define personal identities. A workload token is nonetheless carry-capable for the binding profile's reference form, and the two answers have different trust roots (a workload key versus a named human's key under organizational authority), which is why neither format can absorb the other. RATS (RFC 9334) and the Entity Attestation Token (RFC 9711) attest platform or workload trustworthiness, an orthogonal trust root; the receipt borrows EAT's verifier-relevant-nonce freshness discipline with a higher floor.

Authorization and delegation. OAuth Rich Authorization Requests (RFC 9396) and GNAP (RFC 9635) authorize a client's requested scope; step-up authentication (RFC 9470) can demand fresh human authentication but yields no durable, portable artifact of the approval. Transaction tokens (draft-ietf-oauth-transaction-tokens) propagate context across a machine call chain within a trust domain, short-lived and

online-validated; the receipt is the human-authority root from which such a chain can descend. Delegation-receipt work (draft-nelson-agent-delegation-receipts) binds a *user's* delegation to an *operator's* instructions, upstream; the authorization receipt binds an organizational approver to an exact action downstream, with separation-of-duties and quorum semantics the delegation layer does not formalize, and the two compose by reference. CIBA may serve as the transport by which an approver is reached; the signoff is what the approver produces on arrival. Security Event Tokens (RFC 8417) and CAEP convey issuer assertions that an event occurred, not a human's pre-execution approval bound to one action. Per-hop machine-side route and execution receipts (AgentROA and kin) govern delegated machine authority; none binds a named human, and the receipt's delegation constraints share their tighten-only scope-narrowing discipline. For long-lived evidence, Evidence Record Syntax (RFC 4998) supplies the re-timestamping pattern by which receipts remain verifiable after their original algorithms weaken.

10. Limitations

10.1 What a receipt does not prove

A receipt proves that a key enrolled under an approver identifier signed the canonical bytes of one action, once. It does not prove that a particular natural person exercised the key: identifier-to-person binding is the directory and identity-proofing layer's property (Section 3.5), and possession of the enrolled device plus its user-verification factor is the practical proxy the artifact rests on. A coerced approver, a shoulder-surfed PIN, or a human who has enrolled multiple identities all produce receipts that verify. Distinctness checking operates over enrolled identities; whether two identities are two humans is an enrollment control. Receipts make insider events attributable, named, and evidenced, which raises their cost; they do not make them impossible, and conforming implementations are prohibited from claiming otherwise.

10.2 Presentation attacks: narrowed, not solved

The gravest risk in the protocol is that the approver signs context hash H believing it represents action X when it represents action Y. A signature proves user presence and an act of approval toward *whatever was rendered*; cryptography cannot prove the rendering was faithful. The specification requires escalating mitigations: the signing client must render from the exact bytes that were hashed, never from a separately supplied description; for high-value policies, render templates must be registered with the policy and committed under the policy hash, so display logic is inside what the signature covers; above an organization-designated threshold, material parameters must additionally be rendered on a second surface not authored by the orchestrating operator. Initiator-supplied free text is rendered as untrusted content under a hard length cap, visually separated from the operator-rendered action.

The residual risk is stated without minimization: absent a trusted display path, that is, hardware the operator does not author, rendering fidelity is enforced by controls, audit, and consented mismatch drills, not by mathematics. What the cryptography does provide is exactness of evidence: the receipt contains the full Action Object actually signed, so any divergence between what was displayed and what was executed is detectable after the fact with proof rather than testimony. The evidence side of the what-you-see-is-what-you-sign problem is closed; the perception side is narrowed.

A related human-factors limitation is upstream of any attack: a gate that humans route around protects nothing, and rubber-stamping is the empirical failure mode of human-in-the-loop controls under volume. The protocol is therefore not a general approval workflow; deployments are directed to scope signoff to genuinely high-risk, low-frequency actions and to treat signing-latency floors and zero deny rates as warning signs. The artifact's evidentiary value is only as strong as the attention of the human at its center.

10.3 Log completeness is unprovable offline

Offline verification establishes that a receipt was included in a log tree whose head the operator signed. It cannot establish, offline, that the tree shown to this verifier is the tree shown to everyone (equivocation requires gossip or witness cosigning to detect, which are online activities), nor that the log is complete: the absence of a receipt for some action is not offline-provable, and becomes evidence only through the signed observed-absence mechanism of Section 5, which attests a defined search and its emptiness, never a universal negative. For actions that a policy gates on signoff at an enforcing deployment class, the absence of any valid receipt is evidence that the control was bypassed; that property comes from the gate, not from the log. Revocation currency likewise requires an online consultation of a current directory head and checkpoint. These are boundaries of the offline model, stated as such, rather than defects a future version will quietly remove.

10.4 Scope of the sufficiency layer

An evidence-graph verdict is evidence of sufficiency under a stated, relying-party-owned policy at a stated time. It does not adjudicate disputes, does not establish business correctness, and inherits, without strengthening or weakening, the trust model of each artifact class it consumes. The publisher of these formats is not an auditor, regulator, or insurer; the artifacts are designed to support the determinations of such parties and never to substitute for them.

11. Conclusion

The gap this work addresses is narrow and, we have argued, real: the agent-security stack now produces abundant signed evidence about what an agent *is*, what it was *delegated*, what it *did*, and where that was *logged*, while the specific fact most disputes turn on, that a named accountable human authorized this exact irreversible action before it happened, remains a database row in the custody of an interested party. The authorization receipt makes that one fact portable, offline-verifiable, and one-time-consumable; the binding profile lets adjacent record formats carry it without redefining it; and the evidence-graph layer turns "is this pile of artifacts enough?" into a deterministic, replayable, relying-party-owned computation whose own outcome is evidence.

The design's discipline is subtraction. The operator is removed from the signature path, then prevented from re-entering through directory authority. The presenter is removed from the sufficiency decision. Symmetric primitives are removed from the verification path. Absence is removed from the space of things that can quietly count as consent. What remains is deliberately small: canonical bytes, approver-held keys, single consumption, and proofs that travel inside the artifact.

We have been equally deliberate about the boundary of the claims. The formal models cover the core state machine and the quorum construction and nothing beyond them; the cross-language verifiers are a consistency check, not independence, with an external clean-room reimplementation in progress; rendering fidelity is enforced by controls rather than mathematics; and verification, everywhere in this design, proves signature, binding, and log-inclusion integrity, never that the authorized action was correct. Evidence of authorization is not a verdict on conduct. It is the input that lets whoever must reach such a verdict, an auditor, an insurer, a court, do so from artifacts rather than testimony, and that is the entire ambition of the design.

Artifact Availability

The reference implementation (receipt, quorum, evidence-chain, host-record binding, and evidence-graph layers, with the JavaScript, Python, and Go verifiers), the TLA+ and Alloy models with their checked configurations and proof-status ledger, the 8-suite/58-vector conformance battery with deterministic regeneration scripts, and the worked vectors referenced in this paper (binding, observed-absence, and evidence-graph) are published under the Apache-2.0 license in the EMILIA Protocol repository. The protocol specifications are available as individual Internet-Drafts via the IETF Datatracker (draft-schrock-ep-authorization-receipts and companion documents); Internet-Drafts are working documents and should be cited as work in progress.

References

- I. Schrock, "Authorization Receipts for High-Risk Agent Actions," Internet-Draft draft-schrock-ep-authorization-receipts-05, work in progress, July 2026.
- I. Schrock, "Binding Named-Human Authorization Evidence into Agent-Action Records," Internet-Draft draft-schrock-human-authorization-binding-00, work in progress, July 2026.
- I. Schrock, "Action Evidence Graphs and Evidence Policy Replay for High-Risk Agent Actions (EP-AEG)," Internet-Draft draft-schrock-ep-action-evidence-graph-00, work in progress, July 2026.
- I. Schrock, "Multi-Party Human Authorization (EP-QUORUM)," Internet-Draft draft-schrock-ep-quorum-01, work in progress, June 2026.
- I. Schrock, "Authorization Evidence Chains: Composing Heterogeneous Agent-Authorization Receipts (EP-AEC)," Internet-Draft draft-schrock-ep-authorization-evidence-chain-01, work in progress, July 2026.
- A. Rundgren, B. Jordan, S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020.
- W3C, "Web Authentication: An API for accessing Public Key Credentials, Level 2," April 2021.
- "An Architecture for Trustworthy and Transparent Digital Supply Chains (SCITT)," RFC 9943.
- J. Schaad, "CBOR Object Signing and Encryption (COSE): Structures and Process," RFC 9052.
- "The Entity Attestation Token (EAT)," RFC 9711.
- "Remote Attestation procedureS (RATS) Architecture," RFC 9334.
- "Grant Negotiation and Authorization Protocol (GNAP)," RFC 9635.

- "OAuth 2.0 Rich Authorization Requests," RFC 9396.
- "OAuth 2.0 Step Up Authentication Challenge Protocol," RFC 9470.
- "Security Event Token (SET)," RFC 8417.
- "Evidence Record Syntax (ERS)," RFC 4998.
- OpenID Foundation, "OpenID Connect Client-Initiated Backchannel Authentication Flow, Core 1.0," September 2021.
- R. Nelson, "Delegation Receipt Protocol for AI Agent Authorization," Internet-Draft draft-nelson-agent-delegation-receipts-10, work in progress, June 2026.
- "Transaction Tokens," Internet-Draft draft-ietf-oauth-transaction-tokens, work in progress.